AI Intern Assessment - Ambitio

Document Understanding, Grounded Drafting, and Improvement from Edits

You are joining Ambitio as an AI Engineer. The team needs an internal work flow that ingests messy legal-style documents, pulls usable information out of them, and turns that information into grounded draft outputs an operator can edit.

The inputs will not be clean. Expect scanned pages, low-resolution PDFs, handwritten notes,partially illegible records, and inconsistently formatted files. Your system has to cope with that.

At a high level, the system you build should:

- ingest and process the source documents,
- extract usable text and structured fields,
- retrieve relevant evidence from those documents,
- generate grounded draft responses or legal-style drafts, and
- get better over time by learning from how operators edit the default drafts.

The kind of draft output you choose to generate is up to you. Reasonable picks include:

- a title review summary,
- a case fact summary,
- a notice-related summary
- a document checklist, or
- a first-pass internal memo.

Whatever you choose, the output must be grounded in the underlying documents. We are not looking for confident-sounding text built on unsupported assumptions.

Task Requirements

1. Document Processing

Accept messy legal-style documents and pull useful content out of them. Concretely:

- OCR or text extraction over scanned or noisy files
- reasonable handling of partially unclear inputs
- produce extracted text plus structured data that downstream steps can actually use

The output of this stage should be ready to feed into retrieval and drafting without further cleanup.

2. Grounded Retrieval

Build a retrieval layer over the processed documents so that generation is anchored to actual source material. The retrieval layer should:

- surface the relevant passages for a given drafting task,
- feed that evidence into the generation step, and
- make it possible to inspect which evidence supported which part of the output.

The goal is grounded answering, not generic generation.

3. Draft Generation

Using the processed content and retrieved evidence, generate a draft response or draft legal- style output. A good draft is:

- relevant to the provided documents,
- grounded in retrieved evidence, and
- structured well enough to be useful as a first pass.

We are not evaluating legal correctness. We are evaluating whether the output is well supported by the source material and whether the system around it is designed well.

4. Improvement from Operator Edits

Assume an operator reviews the default draft and edits it. Your system should:

- capture those edits,
- extract something reusable from them, and
- use that signal to make future drafts better.

We are looking for a real improvement loop, not a side-by-side version diff.

What to Submit

Required:

- source code
- README with setup and run instructions
- short architecture overview
- brief write-up of assumptions and tradeoffs
- sample inputs and outputs
- evaluation approach and results

Optional (nice to have, not required):

- API endpoints
- simple UI
- tests
- Docker setup
- Evaluation Rubric (100 Points)
- 1. Document Processing — 25 points
- handling of messy inputs
- OCR / extraction quality
- usefulness of extracted and structured outputs
- whether the extracted output is genuinely usable downstream

2. Retrieval and Grounding — 25 points

- retrieval quality
- relevance of retrieved context
- whether generated outputs are grounded in source material
- whether supporting evidence can be inspected
- how well unsupported generation is controlled
- 3. Draft Quality — 10 points
- usefulness of the generated draft
- clarity and structure
- consistency with the source documents
- overall quality as a first-pass output

4. Improvement from Edits — 25 points

- how edits are captured
- whether reusable patterns are learned
- whether future outputs improve meaningfully
- 5. Code Quality and System Design — 10 points
- code organization
- maintainability
- modularity
- error handling
- scalability of the overall design

6. Documentation and Clarity — 5 points

- ease of understanding
- setup clarity
- quality of explanation and reviewer experience

Notes

- You may use mock or synthetic sample documents.
- You may simulate operator edits if needed.
- Keep the scope practical.
- We care more about engineering quality, grounding, and thoughtful design than visual
- polish.
- Your README, architecture notes, and sample inputs/outputs matter as much as the code
- — don't skip them.
- Scope down where needed — pick the parts you can do well and ship those cleanly.

How to Submit

- Share a git repo and writeup on your approach to
  - shivam.saboo@ambitio.in and Juhi.kumari@ambitio.in
  - With subject: AI Intern Assignment

Reviewer Focus

- A strong submission should clearly show:
- how messy documents are processed,
- how retrieval supports the generated output,
- how the draft stays grounded in source evidence, and
- how operator edits improve future results.